Voice-Controlled KUKA KR4 R600 Industrial Robot

Technical Report — System Architecture, Theory & Mathematics

6-Axis Robot · OpenGL CAD Simulator · Python COM Automation

Coordinate Systems: World · Base · Axis · TCP

Speech-Driven Motion Control

Contents

1. Executive Summary
2. System Overview & Hardware Context
3. Software Architecture & Dependencies
4. Theoretical Foundations: Robot Kinematics
5. The Four Coordinate Systems
6. The Mathematics of Transformations
7. Program Flow — How It Runs
8. Subsystem Deep-Dive (Voice, Motion, State)
9. Key Engineering Problems & Their Solutions
10. Limitations & Future Work

1. Executive Summary

This project implements a **hands-free, voice-controlled motion system** for a KUKA KR4 R600 six-axis industrial robot, visualised through an OpenGL-based CAD simulator. A spoken command such as *"forward"*, *"A3"*, or *"clockwise"* is captured by a microphone, transcribed to text, interpreted, and converted into a precise motion command sent to the robot through Microsoft COM automation.

The system's defining feature is that it supports **four distinct coordinate systems** — World, Base, Axis, and TCP — each of which interprets the *same* spoken word (e.g. "forward") differently depending on the reference frame currently active. The operator selects a mode by voice, performs movements, and can say "switch" to change frames or "halt" to stop. The architecture follows an explicit flowchart in which decision points prompt the user and state transitions are applied silently by the system.

Aspect	Implementation
Input	3-second microphone capture → Google Speech Recognition
Control bridge	Python `win32com.client` → `OpenGLRobot.exe`
Robot	KUKA KR4 R600, 6 revolute joints
Frames	World, Base, Axis (joint), TCP (tool)
Motion primitives	`moveRobotLinear()` (Cartesian), `moveAxis()` (joint)

2. System Overview & Hardware Context

2.1 The KUKA KR4 R600

The KR4 R600 is a small-payload, high-precision industrial manipulator. The two numbers encode its key specs: a payload around 4 kg and a reach of roughly 600 mm. Like virtually all articulated industrial arms, it has **six revolute (rotary) joints**, labelled A1 through A6. This six-degree-of-freedom configuration is the minimum needed to position the tool tip at *any* point in the workspace at *any* orientation — three joints mainly govern position (the "arm") and three mainly govern orientation (the "wrist").

2.2 The Simulator

Rather than driving physical hardware, this project targets `OpenGLRobot.exe`, a 3D CAD simulator that renders the robot and executes the same motion API a real controller would expose. The robot's geometry, joint limits, and home pose are defined in an XML configuration file (`KR4_R600_Stud2.xml`), and the 3D meshes live in a CAD folder. The simulator exposes a COM (Component Object Model) interface so that external programs — here, Python — can command it programmatically.

> **Why a simulator?** It lets the team develop and debug motion logic, coordinate-frame handling, and voice parsing without risk to expensive hardware or people. The control code is written so that the same commands would, in principle, drive a real controller.

3. Software Architecture & Dependencies

The program is a single Python script organised into clear functional layers. Each library plays a specific role:

Library	Role in the system
`win32com.client`	COM bridge — dispatches calls to `OpenGLRobot.exe`
`pythoncom`	Initialises/uninitialises the COM threading environment
`sounddevice`	Records raw audio from the microphone
`speech_recognition`	Sends audio to Google Speech API, returns text
`wave` / `io`	Wraps raw PCM samples into an in-memory WAV file
`math`	Trigonometry for rotation matrices, degree↔radian conversion
`os` / `time`	Process management (kill stale instances) and settle delays

3.1 Layered structure

- **Voice I/O layer** — `listen_once()`, wake-word and keyword detectors.
- **Parsing layer** — converts fuzzy transcribed text into discrete intents (a coordinate system, a tool, an axis number, a direction).
- **Robot abstraction layer** — `init_robot()`, `read_tcp_position()`, `rehome_robot()`, and the movement helpers.
- **Mode controllers** — `run_world_mode()`, `run_base_mode()`, `run_axis_mode()`, `run_tcp_mode()`.
- **Main loop** — wake → init → repeatedly (select frame → run mode → switch/halt).

4. Theoretical Foundations: Robot Kinematics

To understand *why* the code is structured the way it is, one must understand the two fundamental kinematics problems of any robot arm.

4.1 Joint space vs. Cartesian space

A 6-axis robot can be described in two complementary ways:

- **Joint space** — the six joint angles (θ_1, θ_2, …, θ_6). This is what the motors actually control.
- **Cartesian (task) space** — the position (X, Y, Z) and orientation of the tool tip in 3D space. This is what a human cares about.

4.2 Forward kinematics

Forward kinematics answers: *given the six joint angles, where is the tool tip?* It is a deterministic chain of transformations from the base to the tool, multiplied together. In this project, the `moveAxis()` command operates here — you specify joint angles and the simulator computes the resulting pose. This is exactly what **Axis mode** does.

$$T_{\text{tool}}^{\text{base}} = A_1(\theta_1) \cdot A_2(\theta_2) \cdot A_3(\theta_3) \cdot A_4(\theta_4) \cdot A_5(\theta_5) \cdot A_6(\theta_6)$$

Each $\mathbf{A_i}$ is a 4×4 homogeneous transformation matrix describing one link relative to the previous one (typically built from Denavit–Hartenberg parameters). Their product gives the full pose of the tool relative to the base.

4.3 Inverse kinematics

Inverse kinematics answers the harder question: *given a desired tool position (X, Y, Z), what joint angles achieve it?* This generally has multiple solutions (elbow up/down, etc.) or none (if out of reach). In this project, the `moveRobotLinear()` command hides this complexity — you specify a Cartesian target and the simulator's internal solver computes the joint angles. This is what **World, Base, and TCP modes** rely on.

> **Key architectural consequence:** Axis mode is the only mode that commands joints directly. All three Cartesian modes funnel through one primitive, `moveRobotLinear()` — and a crucial discovery (Section 9) is that this primitive *always* interprets its arguments in world coordinates, which forces Base mode to do its own frame mathematics in software.

5. The Four Coordinate Systems

This is the conceptual heart of the project. The same command word means different things in different frames. Imagine you say *"forward"*:

Mode	Reference frame	What "forward" means	API used
World	Fixed global origin of the cell	+X along the world's X axis, always the same direction	`moveRobotLinear`
Base	A user-defined work surface (e.g. a sheet of paper)	+X along the *tilted/rotated* base surface	`moveRobotLinear` + rotation

Axis	Individual joints	N/A — you rotate one joint CW/ ACW instead	`moveAxis`
TCP	The tool tip itself	Forward *relative to where the tool points*	`moveRobotLinear`

5.1 World mode

The simplest frame. Movements are absolute relative to the cell's fixed origin. “Up” is always true vertical. Used as the neutral baseline.

5.2 Base mode

An operator often wants to work relative to a tilted fixture or workpiece. A “base” defines a new origin and orientation (position + Euler angles). When the base is rotated, “forward” should follow the *surface*, not the world. The code stores two predefined bases (e.g. a “Paper” fixture) and rotates each step vector into world coordinates — the mathematics of Section 6.

5.3 Axis mode

Here the operator selects an individual joint (A1–A6) and rotates it clockwise or anticlockwise in fixed 5° steps. This is direct joint-space control — pure forward kinematics. Notably, Axis mode does *not* prompt for a tool, because per the project's flowchart, tool selection belongs exclusively to TCP mode.

5.4 TCP mode

TCP stands for **Tool Center Point** — the working tip of whatever tool is mounted (a pen, a gripper, a welding torch). In TCP mode the operator first selects a tool, whose geometric offset (X, Y, Z + orientation) from the wrist flange is registered via `setTool()`. Movements are then conceptually relative to the tool. The chosen tool persists: when the operator later switches to World/Base/Axis, the same tool is silently re-applied so the physical configuration stays consistent.

6. The Mathematics of Transformations

This section explains every piece of mathematics in the code, building from the basics.

6.1 Degrees and radians

Humans and config files use degrees; trigonometric functions use radians. The conversion factor appears throughout the code as `gr = π / 180`:

$$\theta_{rad} = \theta_{deg} \times (\pi / 180)$$

So a joint angle list in degrees is converted before being sent to `moveAxis()` via `[a * gr for a in axis_angles_deg]`.

6.2 The home pose

The robot's neutral reference configuration is defined in joint space:

$$\text{HOME} = [0°, -90°, 90°, 0°, 45°, 0°]$$

These six numbers are the angles of A1…A6 that place the arm in a known, safe starting posture. Re-homing before every mode guarantees a clean, repeatable reference.

6.3 Rotation matrices — the core of Base mode

A rotation in 3D is represented by a 3×3 orthogonal matrix. The code builds rotations about each principal axis. For a rotation by angle α about the X axis:

$$R_x(\alpha) = \begin{vmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{vmatrix}$$

About the Y axis by angle β:

$$R_y(\beta) = \begin{vmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{vmatrix}$$

About the Z axis by angle γ:

$$R_z(\gamma) = \begin{vmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

6.4 Composing the full rotation

A base's orientation is given by three Euler angles (rx, ry, rz). The code composes them in the order $\mathbf{R} = \mathbf{R_z} \cdot \mathbf{R_y} \cdot \mathbf{R_x}$ (the `mat_mul(mat_mul(Rz, Ry), Rx)` call). Because matrix multiplication is applied right-to-left to a vector, this means: *first* rotate about X, then Y, then Z. This is the standard ZYX (yaw-pitch-roll) Euler convention.

$$R = R_z(\gamma) \cdot R_y(\beta) \cdot R_x(\alpha)$$

> **Why order matters:** Rotations do *not* commute. $R_z \cdot R_x \neq R_x \cdot R_z$. Choosing and consistently applying one convention (here ZYX) is essential, otherwise the base directions would be wrong.

6.5 Transforming a step vector

When the operator says “forward” in Base mode, the desired step is expressed in the *base* frame, e.g. a vector $\mathbf{v}$ = (20, 0, 0) mm. But `moveRobotLinear()` needs world coordinates. So the code multiplies:

$$\mathbf{v}_{world} = R \cdot \mathbf{v}_{base}$$

Written out, each world component is a dot product of a matrix row with the step vector — this is exactly what `transform_step()` computes:

$$\begin{aligned} v_{wx} &= R_{00}d_x + R_{01}d_y + R_{02}d_z \\ v_{wy} &= R_{10}d_x + R_{11}d_y + R_{12}d_z \\ v_{wz} &= R_{20}d_x + R_{21}d_y + R_{22}d_z \end{aligned}$$

The new world target is then the current position plus this rotated step:

$$P_{new} = P_{current} + R \cdot \mathbf{v}_{base}$$

> **Worked example.** Suppose a base is rotated 90° about X (rx = 90°, ry = rz = 0). Then sin90° = 1, cos90° = 0. A “forward” step of (20, 0, 0) is unaffected by an X-rotation (it lies on the rotation axis), so it maps to world (20, 0, 0). But an “up” step of (0, 0, 20) in the base maps to world (0, −20, 0) — what feels like “up” on the tilted surface is actually horizontal motion in the world. This is precisely the correction Base mode provides.

6.6 Why only translation is rotated

The code passes orientation arguments as zeros to `moveRobotLinear(x, y, z, 0, 0, 0)` — it commands position changes while holding tool orientation fixed. Only the 3-element *translation* step needs the rotation transform; the full 4×4 homogeneous machinery is unnecessary for pure jogging, which is why the code uses a compact 3×3 matrix.

7. Program Flow — How It Runs

The main entry point follows this sequence:

1. **COM init** — `pythoncom.CoInitialize()` sets up the threading apartment.
2. **Wake word** — the program loops on the microphone until it hears “hello”/“hallo”.
3. **Robot init** — `init_robot()` kills stale processes, dispatches COM, opens the CAD window, loads the XML, applies a default base/tool, asserts the base frame, and homes the robot.
4. **Main loop** — repeatedly:
   - `select_coordinate_system()` prompts by voice for World/Base/Axis/TCP (or Halt).
   - The matching mode controller runs its own command loop.
   - The mode returns *“switch”* (go back to frame selection) or *“halt”* (exit).
5. **Shutdown** — `destroyRobotWindow()` and `CoUninitialize()`.

7.1 The flowchart discipline

A guiding principle is the distinction between **decision diamonds** and **state boxes** in the project's flowchart:

- A **decision diamond** requires a user prompt (e.g. “which tool?”, “which base?”).
- A **state box** is applied *silently* by the system (e.g. re-applying the active tool when entering World mode).

This is why tool selection only ever prompts in TCP mode, while World/Base/Axis silently re-apply whatever tool was last chosen.

8. Subsystem Deep-Dive

8.1 Voice capture (`listen_once`)

The function records 3 seconds of mono audio at 44.1 kHz, packs the raw 16-bit PCM samples into an in-memory WAV container, and hands it to Google Speech Recognition. The recognised text is lower-cased and stripped. Any failure (silence, network error, no match) returns an empty string, which the calling loops treat as “try again”.

```
# Conceptual flow of listen_once()
recording = sd.rec(duration * fs)      # capture PCM
wav = wrap_as_wav(recording)           # in-memory WAV
text = recognizer.recognize_google(wav)# → "forward"
return text.lower().strip()
```

8.2 Robust keyword parsing

Speech recognition is imperfect, especially for short words. The code therefore matches **phonetic variants**. “Base” might be heard as “bass”, “bays”, “vase”, “pace”, etc., and the parser accepts all of them. Axis numbers are even trickier — “A1” may arrive as “a one”, “hey 1”, “ay won”. The `parse_axis_number()` function handles single-token forms, “A”-sound + number-word pairs, and bare digits.

> **Design insight:** For a short, common vocabulary the safest strategy is broad, forgiving matching rather than exact string equality — a recurring theme in the project's learnings.

8.3 Position management — the single source of truth

A subtle but critical part of the design: the cached Cartesian position (`pos_x, pos_y, pos_z`) must always reflect the *real* tool position after any state change. `read_tcp_position()` tries several possible API getter names, rejects all-zero results (which indicate an uninitialised value), and only as a last resort falls back to a known-good hardcoded home. Reading the live position before the first move prevents the tool from “jumping” unexpectedly.

8.4 Motion helpers

- `move_linear(dx,dy,dz)` — adds a step to the current world position and commands it; updates the cache only after success.
- `move_in_base_frame(dx,dy,dz)` — rotates the step through the active base matrix, then calls `move_linear` .
- `rotate_selected_axis(direction)` — adjusts one joint by ±5°, checking joint limits via `getJointMin/Max` , and rolls back on failure.

9. Key Engineering Problems & Their Solutions

The current code is the product of iterative debugging. The most instructive problems:

Problem	Root cause	Solution
World mode motion failed	`setMoveFrameType('MOVE_FRAME_WORLD')` was unsupported and harmful	`moveRobotLinear` is already world-referenced; the call was removed and the base frame asserted instead
Base directions felt wrong	`setBase()` does *not* change the frame `moveRobotLinear` uses	Transform each step through the base rotation matrix in software (Section 6)
First move "jumped"	Stale / hardcoded TCP position cache	Make live `read_tcp_position()` the primary source with zero-rejection; hardcoded value only as last resort
Axis mode wrongly asked for a tool	Misreading the flowchart — tool selection is TCP-only	Restrict the tool prompt to TCP mode; other modes re-apply silently
"Base" mis-recognised	Short word, many homophones	Broad phonetic variant matching
CAD meshes failed to load	Zombie COM processes from prior runs corrupting new `Dispatch()`	Force-kill `OpenGLRobot.exe`, wait, then re-init; `forceUpdateCad()` after every state change

> **The central lesson:** `moveRobotLinear()` is inherently world-referenced regardless of `setBase()` or frame-type calls. Any frame-relative behaviour the operator expects must be produced explicitly in software — which is exactly why the rotation-matrix mathematics exists in this project.

10. Limitations & Future Work

- **Fixed-duration capture** — the 3-second window can clip long commands or waste time on short ones; voice-activity detection would improve responsiveness.
- **Cloud dependency** — Google Speech requires an internet connection; an offline recogniser would make the system self-contained.
- **Translation-only Cartesian jogging** — orientation is held fixed (zeros passed to `moveRobotLinear`); rotational jogging in Cartesian modes is a natural extension.
- **TCP frame approximation** — if the simulator lacks a true tool frame type, TCP moves may fall back to world; a full tool-frame transform (mirroring Base mode) would make TCP mode geometrically exact.

- **Discrete steps** — motion is in fixed increments (20 mm / 5°); variable or continuous jogging would suit fine positioning.

> This report describes the system as implemented in the integrated four-mode script. The architecture is deliberately modular, so each mode and each subsystem can evolve independently while preserving the flowchart-driven control discipline.